

Problem Set 03

Equipo 1 – Loreana Castaldo, Guillermo González, Hernan López, Federico Martínez y Gerónimo Sosa

Ejercicio 1

1- Pseudocodigo Busqueda en Tries:

Obtenemos la raíz de un árbol, si la misma es nula devolvemos ese valor. De lo contrario, recorremos el árbol en preOrden hasta encontrar la clave con el criterio, si se encuentra un nodo con ese criterio se devuelve, sino de devuelve nulo.

2- Precondiciones:

- El criterio de búsqueda debe ser válido.
- El árbol debe de existir y no estar vacío

Postcondiciones:

- Si existe un nodo con el criterio buscado, devolverlo.
- Si no existe un nodo con el criterio buscado devolver nulo.
- El árbol no puede ser modificado.

3-

```
NodoTrie <T> buscar (String criterio){  
  NodoActual <- this  
  Para cada palabra p de criterio hacer  
    UnHijo <-nodoActual.obtenerHijo(p)  
    Si un hijo es nulo entonces  
      devolver nulo  
    Sino  
      nodoActual= unHijo  
  Fin  
  Fin para  
  Si nodoActual.esHoja entonces  
    Devolver nodoActual  
  Sino  
    Devolver nulo  
  Fin si  
}
```

4-

NodoTrie <T> buscar (String criterio){ $\rightarrow O(n*k)$ $\rightarrow$ (n longitud clave, k cantidad de hijos por nodo.

```
        NodoActual <- this  $\rightarrow O(1)$ 
        Para cada palabra p de criterio hacer  $\rightarrow O(n)$ 
            UnHijo <- nodoActual.obtenerHijo(p)  $\rightarrow O(n)$ 
        Si un hijo es nulo entonces  $\rightarrow O(1)$ 
            devolver nulo  $\rightarrow O(1)$ 
        Sino
            nodoActual= unHijo  $\rightarrow O(1)$ 
        Fin
    Fin para

    Si nodoActual.esHoja entonces  $\rightarrow O(1)$ 
        Devolver nodoActual  $\rightarrow O(1)$ 
    Sino
        Devolver nulo  $\rightarrow O(1)$ 
    Fin si
}
```

FALTAN 5 , 6 y 7

Ejercicio 2

1- Nodo:

- Hijos[26[// referencias a hijos
- EsFinPalabra / booleano
- Paginas / lista de enteros

Ejercicio 5

Describir en lenguaje natural, pre y post condiciones, pseudocódigo de las operaciones y análisis de ordenes de las siguientes operaciones del Trie y NodoTrie:

1. Operación buscar palabra completa.
2. Obtener la lista de palabras por un prefijo dado.
3. Insertar una palabra con un dato asociado.
4. Eliminar una palabra del Trie.

1. Lenguaje natural: Se busca la palabra dada por parámetro recorriendo carácter por carácter desde la raíz. Si en el recorrido no existe el hijo que corresponde a la siguiente letra de la palabra es que la palabra no esta almacenada. Si se logra recorrer toda la palabra y el último nodo esta marcado como fin de la palabra, es decir esPalabra() = True, entonces la palabra esta almacenada.

Precondiciones:

- El árbol Trie tiene que estar inicializado
- La palabra tiene que ser diferente a nulo
- La palabra tiene que conformarse solo con letras minúsculas

Postcondiciones:

- No modifica el árbol
- Retorna un Entry si la palabra existe
- Retorna Null si la palabra no esta almacenada

Pseudocódigo:

COMIENZO

Buscar(palabra) : Entry -> $O(n)$ (n =caractres de la palabra)

NodoActual <- raíz

Para cada caracter c en palabra HACER

Indice <- c - 'a'

SI nodoActual.hijos[indice] es nulo ENTONCES

 Devolver nulo

FIN SI

 NodoActual <- nodoActual.hijos[indice]

FIN PARA

SI nodoActual.esPalabra() == True

 Devolver nodoActual.dato

FIN SI

RETORNAR NULO

2-Lenguaje natural: Se busca la última letra del prefijo y luego se recorren todos los hijos de ese nodo, agregando a la lista todas las palabras encontradas.

Precondiciones:

- El Trie debe de estar inicializado
- El prefijo no puede ser nulo

Postcondiciones:

- No modifica el árbol
- Devuelve una lista con todas las palabras que empiezan con el prefijo

Pseudocodigo:

Predecir(prefijo) : lista $\rightarrow O(n)$

COMIENZO

Resultado<- nuevaLista

NodoActual <- raiz

Para cada caracter c de prefijo HACER

 Indice <- c - 'a'

 SI nodoActual.hijos[indice] es nulo ENTONCES

 Devolver resultado

 FIN SI

 NodoActual<- nodoActual.hijos[indice]

FIN PARA

Recorrer2(nodoActual, prefijo, resultado)

Devolver resultado

FIN

Pseudocódigo

Recorrer2 (nodo, lista) $\rightarrow O(n)$ v

COMIENZO

SI nodo esPalabra = True ENTONCES

Lista.Agregar(nodo.dato)

FIN SI

Para cada hijo h de nodo HACER

 recolectar (lista, hijo)

FIN PARA

FIN

3. Lenguaje Natural: Se recorre letra por letra y si hay alguna letra que no esta se crea en un nodo. Al finalizar el recorrido se marca el último nodo como fin de palabra y se almacena el dato asociado.

Precondiciones:

- El árbol Trie debe de estar inicializado
- La palabra no puede ser nula
- La palabra debe ser válida

Postcondiciones:

- La palabra queda guardada en el árbol
- Si la palabra no existe se crean los nodos necesarios con las letras y se marca el nodo final como palabra devolviendo True
- Si la palabra ya existe no se modifica y retorna false

Pseudocódigo:

Insertar (palabra, dato) : Boleano $\rightarrow O(n)$

COMIENZO

NodoActual $\leftarrow$ raiz

Para cada caracter c en palabra HACER

Indice<- c-'a'

Si nodoActual.hijos[indice] es nulo ENTONCES

nodoActual.hijos[indice] <- nuevo NodoTrie()

FIN SI

NodoActual<- nodoActual.hijos[indice]

FIN PARA

Si nodoActual.esPalabra es falso

nodoActual.dato<- dato

NodoActual.esPalabra <- True

FIN SI

Devolver falso

FIN

- 4- Lenguaje natural: Se corre el árbol buscando la palabra, si la misma existe se desmarca el nodo final como palabra y luego elimina los nodos que no sean fin de palabra.

Precondiciones:

- El árbol debe de estar inicializado
- La palabra debe ser válida

Postcondiciones:

- Si la palabra pertenecía al Trie, eliminarla
- El resto de árbol no se ve alterado

Pseudocódigo

Eliminar (nodo, palabra) -> O(n)

COMIENZO

Si nodo es nulo

Devolver falso

FIN SI

Si palabra es vacía

Si nodo.esPalabra() es falso

devolver falso

FIN SI

Nodo.esPalabra() <- falso

Devolver nodo.tieneHijos()

FIN SI

C<- primer caracter de palabra

Indice <- c - 'a'

Resto <- palabra sin el primer caracter

SI eliminar(hijo, resto)

Eliminar referencia al hijo

Devolver

Nodo.noTieneHijos()

Y nodo.esPalabra es falso

FIN SI

Devolver falso

FIN

Ejercicio 11

Se desea implementar un programa que, en forma eficiente, permita obtener las frecuencias de ocurrencias de las palabras de un libro. Para ello se debe entonces:

1. Seleccionar la o las clases de la API de colecciones de JAVA más apropiada para esta aplicación. Justificar la elección.
2. Dado un archivo de entrada «libro.txt», desarrollar un programa JAVA utilizando la API de colecciones que permita saber las frecuencias de ocurrencias de las palabras del libro.
3. Graficar los resultados de las 10 palabras que más ocurren.

1. Estructura de datos seleccionada: `HashMap<String, Integer>`

Se utilizó `HashMap` para almacenar las frecuencias de las palabras, donde la clave es la palabra (`String`) y el valor es su cantidad de apariciones (`Integer`).

La elección se debe a que las operaciones principales del problema son búsqueda e inserción, las cuales en `HashMap` tienen complejidad promedio $O(1)$. Esto permite contar ocurrencias de forma eficiente incluso para archivos grandes.

No se utilizó `TreeMap` porque no era necesario mantener las palabras ordenadas durante el conteo, y estructuras como `ArrayList` o `LinkedList` resultarían ineficientes al requerir búsquedas lineales $O(n)$ para actualizar cada frecuencia.

El ordenamiento se realiza únicamente al final del procesamiento para obtener las 10 palabras más frecuentes.

2. Procesamiento del archivo

El programa lee `libro.txt` línea por línea utilizando `FileUtils.leerLineas()`.

Cada línea:

- se convierte a minúsculas,
- elimina caracteres no alfabéticos mediante expresiones regulares (`[^\p{L} ]`),
- se divide en palabras utilizando espacios como separadores.

Las frecuencias se almacenan en un HashMap, incrementando el contador correspondiente para cada palabra encontrada. Finalmente, las entradas del mapa se ordenan de forma descendente según su frecuencia para obtener el Top 10.

3. Gráfico de resultados

Los resultados se representan en consola mediante un gráfico ASCII con el formato:

```
palabra      - ##### (frecuencia)
```

La longitud de cada barra es proporcional a la frecuencia máxima encontrada, utilizando un máximo de 40 caracteres para mantener una visualización uniforme.

Ejercicio 13

Parte 1

Investiga cómo está implementado el método `hashCode` en Java para objetos de la clase `Object`. Luego investiga cómo se implementa el mismo para las clases `Integer` y `String`. Explica por qué la implementación es diferente.

Parte 2

Investiga y diagrama cómo son las estructuras internas de un `HashMap`. Con lo investigado en el ejercicio anterior, diagrama el estado de las estructuras luego de insertar las siguientes strings:

1. Implementación de **hashCode** en Java

Object.hashCode()

La implementación por defecto de `Object` genera un hash asociado a la identidad del objeto en memoria. Dos objetos distintos tendrán normalmente hashes distintos, incluso si contienen la misma información.

Integer.hashCode()

En `Integer`, el `hashCode()` corresponde directamente al valor entero almacenado. Por ejemplo:

```
Integer.valueOf(42).hashCode() == 42
```

Esto tiene sentido porque la identidad lógica de un `Integer` es su valor numérico.

String.hashCode()

La clase String calcula el hash en función de todos los caracteres de la cadena utilizando una fórmula polinómica basada en multiplicaciones por 31. De esta forma, dos strings iguales generan siempre el mismo hash.

Las implementaciones son diferentes porque cada clase tiene una definición distinta de igualdad. Object utiliza la identidad del objeto en memoria, Integer depende del valor numérico almacenado y String depende de la secuencia de caracteres de la cadena. En todos los casos se respeta el contrato general de Java, si dos objetos son iguales según equals(), entonces deben generar el mismo hashCode().

2. Estructura interna de **HashMap**

Internamente, **HashMap** utiliza un arreglo de buckets. Cada posición puede almacenar uno o más nodos en caso de colisión.

Cada nodo contiene: clave, valor, hash, y referencia al siguiente nodo.

El índice del bucket se calcula a partir del **hashCode()** de la clave.

Resultados obtenidos:

Hola -> hashCode: 2255068 bucket: 12

HolaMundo -> hashCode: -1191400619 bucket: 5

HashMap -> hashCode: -1932803762 bucket: 14

Colecciones -> hashCode: -1872965711 bucket: 1

No se produjeron colisiones para las strings analizadas utilizando capacidad 16.

3. Implementación de **equals()** y **hashCode()** en **Alumno**

Se implementaron los métodos **equals()** y **hashCode()** utilizando el atributo **id** como criterio de identidad lógica del alumno.

```
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null || getClass() != obj.getClass()) {
```

```

        return false;
    }
    Alumno otro = (Alumno) obj;
    return this.id == otro.id;
}

@Override
public int hashCode() {
    return Objects.hash(id);
}

```

La implementación cumple el contrato general de Java, ya que objetos iguales generan el mismo hash y este permanece consistente mientras no cambie el **id**.

Las pruebas realizadas con **HashSet** y **HashMap** verifican el correcto funcionamiento de la implementación.

Ejercicio 14

Dado un mismo conjunto de claves enteras, insertar las claves en tablas hash usando diferentes estrategias de resolución de colisiones:

- Sondeo lineal.
- Sondeo cuadrático.
- Encadenamiento separado.

Utilizar el siguiente conjunto de claves, en el orden indicado:

45, 12, 37, 82, 29, 54, 31, 76, 18, 93, 11, 68

Se pide:

1. Definir un tamaño de tabla adecuado y justificar la decisión.
2. Definir una función hash sencilla para las claves dadas.
3. Mostrar el estado final de la tabla para cada estrategia.
4. Calcular la cantidad total de colisiones en cada caso.
5. Calcular el promedio de comparaciones para búsquedas exitosas.
6. Calcular el promedio de comparaciones para búsquedas no exitosas, indicando claramente qué claves se utilizaron para probarlas.
7. Comparar los resultados obtenidos y concluir cuál estrategia funcionó mejor para este conjunto de datos.

1. Tamaño de tabla y función hash

Son 12 claves. Siguiendo la recomendación de dimensionar la tabla un 10% más grande: $12 / 0.9$ es aprox 14. El primo más cercano mayor a 14 es $M = 17$.

2. Hash de cada clave

Clave	$H(K) = K \bmod 17$
45	11
12	12
37	3
82	14
29	12 colisión
54	3 colisión
31	14 colisión
76	8
18	1
93	8 colisión
11	11 colisión
68	0

3. Sondeo lineal

Posición	Clave	Notas
0	68	directo
1	18	directo
2	—	
3	37	directo
4	54	colisión en 3, va a 4
5	—	
6	—	
7	—	
8	76	directo
9	93	colisión en 8, va a 9

10	—	
11	45	directo
12	12	directo
13	29	colisión en 12, va a 13
14	82	directo
15	31	colisión en 14, va a 15
16	11	colisión en 11, 12, 13, 14, 15, va a 16

Colisiones totales: 6 claves colisionaron. La clave 11 necesitó 5 intentos extra siendo el peor caso.

4. Sondeo cuadrático — $h(i) = (h_0 + i^2) \bmod 17$

Posición	Clave	Notas
0	68	directo
1	18	directo
2	—	
3	37	directo
4	54	colisión en 3, $(3+1)=4$
5	—	
6	—	
7	—	
8	76	directo
9	93	colisión en 8, $(8+1)=9$
10	11	colisión en 11, 12, 15, 3 $\rightarrow (11+16) \bmod 17 = 10$
11	45	directo
12	12	directo

13	29	colisión en 12, $(12+1)=13$
14	82	directo
15	31	colisión en 14, $(14+1)=15$
16	—	

La clave 11 necesitó 4 intentos en vez de 5 que costó en el lineal.

5. Encadenamiento separado

Posición	Lista
0	68
1	18
3	37 y 54
8	76 y 93
11	45 y 11
12	12 y 29
14	82 y 31

Colisiones totales = 5. No hay agrupamiento.

6. Promedio de comparaciones de las búsquedas que fueron exitosas

Sondeo lineal:

Clave	Comparaciones
68, 18, 37, 82, 76, 45, 12	1 cada una
54, 29, 93, 31	2 cada una
11	6

Promedio lineal = $(7 \times 1 + 4 \times 2 + 6) / 12 = 21/12$ es aprox 1.75

Sondeo cuadrático:

Igual que lineal salvo la clave 11 que necesita 5 comparaciones en vez de 6.

Promedio cuadrático = $(7 \times 1 + 4 \times 2 + 5) / 12 = 20/12$ es aprox 1.67

Encadenamiento separado:

Las 7 claves sin colisión necesitan 1 comparación, las 5 con colisión necesitan 2.

Promedio encadenamiento = $(7 \times 1 + 5 \times 2) / 12 = 17/12$ es aprox 1.42

7. Promedio de comparaciones de las búsquedas no exitosas

Claves de prueba: 10, 20, 50.

Sondeo lineal:

$h(10)=10$ vacía tomó una 1 comparación

$h(20)=3$ recorre 3, 4, 5 vacía, tomó 3 comparaciones

$h(50)=16$ vacía tomó 1 comparación

Promedio ≈ 1.67

Sondeo cuadrático: mismo resultado

Promedio ≈ 1.67

Encadenamiento:

$h(10)=10$ lista vacía, tomó 0 comparaciones

$h(20)=3$ recorre 37, luego 54 sin encontrar, tomó 2 comparaciones

$h(50)=16$ vacía, tomó 0 comparaciones

Promedio ≈ 0.67

Conclusión

El encadenamiento separado funcionó mejor en todos los casos, con el promedio de comparaciones más bajo tanto en búsquedas exitosas como no exitosas. El sondeo cuadrático le ganó al lineal porque tiene menos agrupamiento, lo que se nota en claves como el 11 que colisionó varias veces. Los tres métodos andan bien con un factor de carga de 0.71, pero si subiera el factor de carga el lineal se degradaría mucho más rápido que los otros dos.

Ejercicio 7

Parte 1

Instrucciones de uso de las clases para medición

El sistema de medición permite evaluar el rendimiento de diferentes estructuras de datos vistas en el curso obteniendo el tiempo de ejecución, consumo de memoria y la exportación de resultados a un archivo CSV.

El flujo se realiza en la clase Main y sería el siguiente:

- Instanciar estructuras
- Cargar datos desde la carpeta de recursos brindada en el proyecto
- Instanciar clase Medible
- Ejecutar método medir()
- Obtener y mostrar resultados

Instancias de estructuras

```
Trie<String> trie = new Trie();  
LinkedList<String> linkedList = new LinkedList<>();  
ArrayList<String> arrayList = new ArrayList<>();  
Map<String, String> hashMap = new HashMap<>();  
Map<String, String> treeMap = new TreeMap<>();
```

Cargar datos desde archivos a estructuras

```
List<String> palabrasParaAgregar = new LinkedList<>();  
List<String> palabrasParaBuscar = new LinkedList<>();  
FileUtils.leerLineas(path: "./ut03/listado-general-desordenado.txt", palabrasParaAgregar::add);  
FileUtils.leerLineas(path: "./ut03/listado-general-palabrasBuscar.txt", palabrasParaBuscar::add);
```



```

for (String p : palabrasParaAgregar) {
    // insertar la palabra p en el trie
    trie.insertar(p, p);
    // insertar la palabra p en el linkedList
    linkedList.add(p);
    // insertar la palabra p en el arrayList
    arrayList.add(p);
    // insertar la palabra p en el hashMap
    hashMap.put(p, p);
    // insertar la palabra p en el treeMap
    treeMap.put(p, p);
}

```

Se crean las listas de mediciones

```

List<Medible<List<String>>> medibles = new LinkedList<>();
medibles.add(new MedicionBuscarLinkedList(linkedList));

```

Se ejecuta el método medir para la medición de la estructura y se exportan los datos a un archivo de salida

```

StringBuilder sb = new StringBuilder();
sb.append("algoritmo,tiempo,memoria\n");

for (Medible<List<String>> m : medibles) {

    Medicion mi = m.medir(REPETICIONES, palabrasParaBuscar);
    mi.print();
    sb
        .append(mi.toCSV())
        .append("\n");
}

FileUtils.escribirLineas(path: "./salida.csv", sb.toString());
}

```

Parte 2

```

for (String p : palabrasParaAgregar) {
    // insertar la palabra p en el trie
    trie.insertar(p, p);
    // insertar la palabra p en el linkedList
    linkedList.add(p);
    // insertar la palabra p en el arrayList
    arrayList.add(p);
    // insertar la palabra p en el hashMap
    hashMap.put(p, p);
    // insertar la palabra p en el treeMap
    treeMap.put(p, p);
}

```

Parte 3

```

private static final int REPETICIONES = 20;

```

-ArrayList

```

////////// Ejercicio 7 - Parte 3//////////
ArrayList<Medible<List<String>>> arrayList2 = new ArrayList<>();
arrayList2.add(new MedicionBuscarArrayList(arrayList));
StringBuilder sbArray = new StringBuilder();
sbArray.append("algoritmo,tiempo,memoria\n");
for (Medible<List<String>> m : arrayList2) {
    Medicion mi = m.medir(REPETICIONES, palabrasParaBuscar);
    mi.print();
    sbArray.append(mi.toCSV()).append("\n");
}
FileUtils.escribirLineas(path: "./salidaArray.csv", sbArray.toString());

```

-Trie

```

ArrayList<Medible<List<String>>> arrayTrie = new ArrayList<>();
arrayTrie.add(new MedicionBuscarTrie(trie));
StringBuilder sbArrayTrie = new StringBuilder();
sbArrayTrie.append("algoritmo,tiempo,memoria\n");
for (Medible<List<String>> m : arrayTrie) {
    Medicion mi = m.medir(REPETICIONES, palabrasParaBuscar);
    mi.print();
    sbArrayTrie.append(mi.toCSV()).append("\n");
}
FileUtils.escribirLineas(path: "./salidaTrie.csv", sbArrayTrie.toString());
;

```

-HashMap

```

ArrayList<Medible<List<String>>> hashList = new ArrayList<>();
hashList.add(new MedicionBuscarHashMap(hashMap));

StringBuilder sbHash = new StringBuilder();
sbHash.append("algoritmo,tiempo,memoria\n");

for (Medible<List<String>> m : hashList) {
    Medicion mi = m.medir(REPETICIONES, palabrasParaBuscar);
    mi.print();
    sbHash.append(mi.toCSV()).append("\n");
}

FileUtils.escribirLineas(path: "./salidaHashMap.csv", sbHash.toString());
System.out.println("Medicion HashMap realizada");

```

-TreeMap

```

ArrayList<Medible<List<String>>> treeList = new ArrayList<>();
treeList.add(new MedicionBuscarTreeMap(treeMap));

StringBuilder sbTree = new StringBuilder();
sbTree.append("algoritmo,tiempo,memoria\n");

for (Medible<List<String>> m : treeList) {
    Medicion mi = m.medir(REPETICIONES, palabrasParaBuscar);
    mi.print();
    sbTree.append(mi.toCSV()).append("\n");
}

FileUtils.escribirLineas(path: "./salidaTreeMap.csv", sbTree.toString());
System.out.println("Medicion TreeMap realizada");

///// Ejercicio 7 - Parte 4
List<Medible<List<String>>> mediblesTabla = new ArrayList<>();

```

Parte 4

```
////// Ejercicio 7 - Parte 4
List<Medible<List<String>>> mediblesTabla = new ArrayList<>();

mediblesTabla.add(new MedicionBuscarLinkedList(linkedList));
mediblesTabla.add(new MedicionBuscarArrayList(arrayList));
mediblesTabla.add(new MedicionBuscarHashMap(hashMap));
mediblesTabla.add(new MedicionBuscarTreeMap(treeMap));
mediblesTabla.add(new MedicionBuscarTrie(trie));

StringBuilder sbTabla = new StringBuilder();
sbTabla.append("Estructura,Memoria (MB),Tiempo (ms)\n");

for (Medible<List<String>> m : mediblesTabla) {
    Medicion mi = m.medir(REPETICIONES, palabrasParaBuscar);
    double memoriaMB = mi.getMemoria() / (1024.0 * 1024.0);
    double tiempoMS = mi.getTiempoEjecucion() / 1_000_000.0;
    mi.print();
    sbTabla.append(mi.getTexto()).append(",").append(String.format("%.2f", memoriaMB)).append(",")
        .append(String.format("%.2f", tiempoMS)).append("\n");
}
FileUtils.escribirLineas(path: "./tablaFinal.csv", sbTabla.toString());
System.out.println("Mediciones completas. Archivo tablaFinal.csv generado.");
```

Resultado

```
tablaFinal.csv
1  Estructura,Memoria (MB),Tiempo (ms)
2  MedicionBuscarLinkedList - 0,89 - 8332,73
3  MedicionBuscarArrayList - 0,89 - 3875,48
4  MedicionBuscarHashMap - 1,27 - 3,40
5  MedicionBuscarTreeMap - 1,27 - 6,07
6  MedicionBuscarTrie - 11,92 - 3,44
7
8
```

En términos de Tiempo la estructura Trie presenta ventajas sobre las otras estructuras probadas, mientras que en memoria las estructuras que tienen un mejor desempeño

son las ArrayList y las LinkedList. El archivo de salida tuvo que cambiar el formato de “,” por “-” para mejor visualización de los resultados, rompiendo así el formato CSV.

Parte 5

```
private static final int REPETICIONES = 20;
```

```
/// Ejercicio 7 - Parte 5 ///
```

```
mediblesTabla.add(new MedicionPrededirTrie(trie));
mediblesTabla.add(new MedicionPrededirLinkedList(linkedList));
mediblesTabla.add(new MedicionPrededirHashMap(hashMap));

StringBuilder sbP = new StringBuilder();
sbP.append("Estructura,Memoria (MB),Tiempo (ms)\n");

for (Medible<List<String>> m : mediblesTabla) {

    Medicion mi = m.medir(REPETICIONES, palabrasParaBuscar);

    double memoriaMB = mi.getMemoria() / (1024.0 * 1024.0);
    double tiempoMS = mi.getTiempoEjecucion() / 1_000_000.0;

    mi.print();

    sbP.append(mi.getTexto()).append(",").append(String.format("%.2f", memoriaMB)).append(",")
        .append(String.format("%.2f", tiempoMS)).append("\n");
}

FileUtils.escribirLineas(path: "./tablaPrediccion.csv", sbP.toString());
System.out.println("Table prediccion generada en tablaPrediccion.csv");
```

Resultado

	tablaPrediccion.csv
1	Estructura, Memoria (MB), Tiempo (ms)
2	MedicionBuscarLinkedList - 0,89 - 5037,12
3	MedicionBuscarArrayList - 0,89 - 3258,20
4	MedicionBuscarHashMap - 1,27 - 2,18
5	MedicionBuscarTreeMap - 1,27 - 8,31
6	MedicionBuscarTrie - 11,92 - 2,36
7	MedicionPredecirTrie - 11,92 - 2,32
8	MedicionPredecirLinkedList - 0,89 - 45,58
9	MedicionPredecirHashMap - 1,27 - 150,63
10	

En esta tabla de resultados se puede observar que en relación a los recursos de tiempo y memoria, en lo que es el método de buscar, la estructura más beneficiosa es la de HashMap. Mientras que para la predicción la estructura más óptima es la de Trie. Se vuelve a evidenciar que según las operaciones que un sistema requiera son las estructuras que se deben de elegir para su implementación, ya que algunas presentan mejores ventajas y desventajas frente a otras, no hay ninguna estructura que sea óptima para todos los casos.

El archivo de salida tuvo que cambiar el formato de “,” por “-” para mejor visualización de los resultados, rompiendo así el formato CSV.

Ejercicio 15

Parte 1

El atributo que debería determinar la identidad lógica de un Libro es isbn. Ya que el ISBN es un identificador único asignado a una edición concreta de un libro. Por lo tanto, dos objetos Libro con el mismo isbn deberían considerarse el mismo, aunque sean instancias distintas en memoria. No conviene usar título, autor o año como identidad porque pueden repetirse.

Parte 3

El error conceptual es que se rompe el contrato general entre equals y hashCode. Si dos objetos son iguales según equals, entonces deben devolver el mismo valor de hashCode. Pero si hashCode usa título, podrían tener códigos hash distintos. En un HashSet o HashMap, esto puede provocar que ambos objetos terminen en ubicaciones internas distintas, haciendo que la estructura no detecte que representan el mismo libro.

Parte 5

Si equals y hashCode están correctamente implementados, el HashSet debería conservar un solo elemento. (Está demostrado en el main) Esto ocurre porque HashSet usa primero el hashCode para ubicar el objeto en una zona interna y luego usa equals para determinar si ya existe un elemento equivalente.